Agentic Context Management System

Component Architecture — Living Design Document

This document captures the evolving architecture of a context management system for Claude Code (and other agents). It is structured in three parts: **Part 1** — the inception design (the full theoretical system as originally conceived). **Part 2** — the practical solution (the revised build philosophy focused on what is genuinely missing). **Part 3** — Phase 0 OSS architecture, reflecting what actually ships in v0.2.0. A new closing section (**OSS reality & architecture follow-ups**) catalogues the deltas between the spec and the shipped code, and the architectural work the OSS build surfaced.

Last updated: 2026-05-08 · Reflects robrain repo at commit `a9f803d` on `main`.

PART 1 — High-level idea: inception design

The original full-system conception — five bespoke components designed to solve context management end to end. Preserved here as the theoretical foundation. The practical build decisions that evolved from this are in Part 2; the actually-shipped OSS architecture is in Part 3.

System overview

The system was conceived as five components working together to give Claude Code persistent, causally-structured memory across sessions:

Component	Role	Original location
Sensing	Observes live sessions; detects new vs changed information	With Claude Code (MCP)
Localization	Structural code index; knows what files exist and their shape	With Claude Code (MCP)
Perception	Stores decisions, rationale, rejected options, bug history	External agent
Planning	Decides what context is relevant; ranks memories by task	External agent
Control	Injects context into Claude Code at the right moment	With Claude Code (MCP)
Sharing	Cross-cutting: controls scope (user / local / team / global)	Cross-cutting layer

> **Reality in OSS (v0.2.0, May 2026)**
>
> Of these six, the OSS package (v0.2.0) ships **Sensing**, **Perception (self-hosted)**, the **shared schema**, and the **CLI** as the user-visible surface. **Localization** collapsed into an in-Sensing 'files Claude touched this session' map (no Cursor / Copilot adapters yet). **Planning** and **Control** are cloud-only — none of the originally-planned `control_*` MCP tools exist in OSS. **Sharing** exists only as a `scope` column with default `'team'`; no scope-inference rules run yet.

System architecture diagram

Originally: Sensing and Localization sit inside the Claude Code environment (left boundary), Perception and Planning are external agents (right boundary), and Control bridges both sides. The Sharing layer runs vertically across all components.

Figure 1 — Full system architecture. The OSS shipped surface is documented in Part 3 (Phase 0).

Control component (inception)

Control is the execution layer — the only component that directly touches Claude Code and the user. It receives ranked memories from Planning and a task-boundary signal from Sensing, then injects precisely the right context at the right moment. It is also the point where feedback flows back into the system.

Position in the system

Field	Value
Receives from	Planning (ranked, scored memories) · Sensing (task boundary signal)
Sends to	Claude Code context window · Perception (relevance score feedback) · Causal graph (user corrections)
Lives	Inside Claude Code environment as an MCP server
Key constraint	Must not cost more context than it saves — every injected token is borrowed from Claude's reasoning budget

Data flow through Control — four sequential gates

- **Stage 1 — Token budget + relevance filter.** Planning scores every candidate memory by relevance; Control enforces a hard cap of 500 tokens on total injected context. Memories are ranked by score and truncated from the bottom — lowest-scored memories are dropped first.
- **Stage 2 — Provenance layer.** Every memory that survives the token budget gate is wrapped with metadata: source session identifier, storage date, and a confidence score (high / medium / low). This metadata is visible to Claude and to the user.
- **Stage 3 — Three-tier injection.** Surviving, annotated memories are injected via one of three mechanisms (always-on, task-triggered, on-demand) depending on their priority and the current session state.
- **Stage 4 — Conflict framing.** Any injected memory that directly constrains or contradicts a plausible action is framed as an explicit question Claude must acknowledge: 'Previously X was decided — does that apply here?' Claude's disagreement is itself a signal routed back to Perception.

Three-tier injection detail

Tier	Mechanism	Token budget	When it fires	Design rationale
Always-on	3-line summary injected into the system prompt at session start	≤ 80 tokens	Every session, always	Gives Claude a minimum viable project orientation at near-zero cost. Written by the Memory Writer after each session.
Task-triggered	Relevant memories injected when Sensing detects a topic shift	≤ 300 tokens	On task boundary signal from Sensing	Right moment, right context. Fired between tasks not mid-thought. Framed as prior context, not instruction.
On-demand tool	`get_context(query)` MCP tool Claude can call when it needs to go deeper	≤ 500 tokens per call	When Claude explicitly needs more context	Pull model avoids pushing irrelevant context speculatively.

Drift from original spec

None of the three Control tiers exist in OSS. The OSS user-experience equivalents are: (a) **always-on** → a 3-line summary regenerated by Perception's `POST /projects/:id/regenerate-summary`, fetched on-demand by the CLI but *not* auto-injected; (b) **task-triggered** → does not fire — the OSS user runs `npx robrain inject --query …` manually; (c) **on-demand** → satisfied by the same CLI inject command, plus `robrain explain <file> --why` for file-scoped retrieval. The new `robrain export-memory` command bridges to Claude's native Auto-memory and approximates always-on at file-system level.

Feedback loop

- **Implicit relevance signal** — Sensing watches Claude's reply for terms from the injected context. Matches score the memory up; no matches score it down. Planning uses these scores to improve future relevance ranking.
- **User correction path** — When a user identifies an injected memory as wrong or outdated, Control routes that correction to Perception, which updates the causal graph.
- **Claude disagreement path** — When conflict framing causes Claude to explicitly disagree with prior context, that disagreement is captured as a correction candidate.

Reality in OSS (v0.2.0, May 2026)

Reply scoring **is** implemented in OSS — Sensing computes the term-match locally and POSTs it to Perception's `/scores` endpoint, which updates `historical_relevance` on the decision row (`+0.05` if term match > 0.3, `-0.02` otherwise). User correction routes via `POST /corrections` — surfaced through `robrain review`'s edit / reject actions. Claude-disagreement path is not implemented in OSS (no Control to detect it).

Unresolved risks (Control component, inception)

Failure point	Status	Remaining gap
Injection mechanism	Solved (cloud)	Always-on summary auto-write shipped server-side; OSS surfaces it via CLI inject only.
Timing	Solved (cloud)	Sensing's topic-shift classifier ships in OSS; auto-inject still cloud-only.
Conflict with Claude	Partial	Framing as a question improves engagement but does not guarantee acknowledgment. No enforcement mechanism in the prompt yet.
Context overload	Solved (cloud)	Planning's relevance scorer is cloud-only; OSS inject orders by recency + file-overlap as a coarser proxy.
Stale / wrong memory	Partial	Approval gate (`reviewed_at`) shipped in OSS — see Part 3. Auto-detection of staleness against code diff still TBD.
No feedback loop	Partial	Term-matching shipped in OSS via `/scores`. Semantic comparison upgrade still pending.

Sensing component (inception)

Sensing is the most critical component to get right because everything downstream depends on it. If Sensing misses a decision event or fires the wrong signal at the wrong moment, Perception never gets the data and the whole system fails silently. It sits inside Claude Code as an MCP server and must never add perceptible latency to the user's session.

Position in the system

Field	Value
Receives from	Live session stream (every user message + Claude reply)
Sends to	Perception (decision signals) · Control (task boundary signal) · Perception (reply relevance scores)
Lives	Inside Claude Code environment as a passive MCP server
Key constraint	Must never block the session hot path — all processing is async via a local buffer

Internal architecture — three sequential layers

- **Layer A — stream buffer.** Receives every session turn in real time and writes it to a local buffer with a timestamp. Never blocks; no processing, no classification, no network calls.
- **Layer B — classifier (async, off hot path).** Reads from the buffer and runs three classifiers in parallel: decision classifier (did a meaningful choice get made?), topic-shift classifier (did the task change?), and reply scorer (did Claude use the injected context?).
- **Layer C — router (stateless).** Routes classifier output to the correct destination: decision signals → Perception, task-boundary signals → Control, relevance scores → Perception.

> **Reality in OSS (v0.2.0, May 2026)**
>
> All three layers ship in OSS Sensing as designed: `packages/sensing-mcp/src/buffer.ts` (Layer A), `packages/sensing-mcp/src/classifiers/index.ts` (Layer B — three classifiers), and `packages/sensing-mcp/src/router.ts` (Layer C). The router POSTs decision and flush signals to Perception `/signals`, with the project id pinned via the `X-Project-Id` header (the Bug 3 fix referenced later in this doc).

Flush-on-close hook

Buffer loss is the most dangerous failure mode. If the session closes while the classifier is still processing buffered turns, those turns are gone permanently. The fix: when Claude Code signals session end, Sensing gets a 2-second grace window to drain the buffer. Any unprocessed turns are shipped raw to Perception with a `needs_classification` flag. Perception runs classification offline. Nothing is lost even if the classifiers have not finished.

> **Reality in OSS (v0.2.0, May 2026)**

Shipped. `sensing_end_session` in `packages/sensing-mcp/src/index.ts` uses `Promise.race` against a configurable grace window (default 2 s); on timeout it returns and lets the classifier finish in the background. The schema column `session_turns` + `decisions.needs_classification` backs the raw-flush path.

Sensing — classifier design

Each classifier is designed independently so it can be tuned and upgraded without touching the others. All three run on the same buffered turn input. False negatives are worse than false positives — tune each classifier to be over-sensitive rather than under-sensitive, and let Control's token budget absorb the occasional over-injection.

Classifier 1 — decision classifier

Stage	Mechanism	When it runs	Output
Stage 1 heuristic	Keyword scan for: *decided, rejected, instead of, going with, too slow, we tried, switched to, won't work*; plus implicit-decision proxies (library imports, package-manager invocations, brief commits).	Always — synchronous, < 1 ms	hit / no-hit
Stage 2 LLM confirm	Haiku reads the full turn. Prompted: 'Was a technical decision made here? Output: decision | no-decision + confidence 0–1'.	Only on keyword hit — async ~300 ms	decision_type + confidence
Skip path	If no keyword hit, skip LLM entirely. Mark as no-decision with confidence 0.9.	On no-hit — saves ~95% of LLM calls	no-decision 0.9

Reality in OSS (v0.2.0, May 2026)

Implemented in `classifiers/index.ts`. The OSS keyword list and implicit-decision proxies match the spec; the Stage 2 Haiku prompt is the OSS basic prompt (the calibrated prompt with few-shot negatives is the cloud differentiator — see Part 3 OSS-vs-cloud table).

Classifier 2 — topic-shift classifier

Stage	Mechanism	Threshold	Output
Stage 1 embedding delta	Embed current user message. Compute cosine distance against the last 3 messages.	$\theta = 0.35$ (start). Tune based on FP rate.	distance score
Stage 2 file context check	Did the files being discussed change? Cross-reference Localization's index. File shift AND embedding shift = high-confidence boundary.	Both signals required for high confidence	confidence : high / med / low

Reality in OSS (v0.2.0, May 2026)

Stage 1 ships in OSS with configurable `topicShiftWindowSize`, `topicShiftThreshold`, and a kill-switch `topicShiftDisableEmbedding`. Stage 2's Localization 'index' is implemented as a session-local map of files Claude has touched — no Cursor/Copilot adapter yet. Topic-shift result is returned in the `sensing_record_turn` response so the agent (or, in cloud, Control) can decide what to do with it; in OSS no auto-action is taken.

Classifier 3 — reply scorer

Stage	Mechanism	Version	Output
Stage 1 term match	Check if key terms from the injected memory appear in Claude's reply. Simple string matching.	v1 — shipped in OSS	score 0.0–1.0 (noisy)
Stage 2 semantic check	Embed the injected memory and the relevant portion of Claude's reply; compute cosine similarity.	v2 — when miss-rate data shows need (cloud)	score 0.0–1.0 (reliable)

Drift from original spec

The original 'Bug 2 fix' wording said reply scoring 'moved server-side' to Perception. More precisely: term-match logic still runs in Sensing (`classifiers/index.ts`); only the persistence of the score and the relevance update moved to Perception's `POST /scores`. Sensing now sends `injected_memory_ids` + the computed score, and Perception updates `historical_relevance`. The bug being fixed (empty-string scoring against missing decision text) is closed.

Build plan — v1 vs v2

Classifier	v1 (ship now)	v2 trigger
Decision	Heuristic keyword scan only. No LLM.	Miss rate > 15%
Topic shift	Embedding delta only, $\theta = 0.35$.	FP rate > 10%
Reply scorer	Term matching only.	Score distribution too noisy
Flush-on-close	Always — non-negotiable from day 1.	N/A — never remove

Sensing — open issues from inception

Issue	Severity	Status (OSS v0.2.0)
Localization fallback behavior	Partial	Resolved as 'session-local files-touched map.' Cursor/Copilot adapters still TODO.
Session-open hook	Partial	`sensing_start_session` warm-fetches the always-on summary; embedding model warm-up is implicit on first turn.
Scope tagging at capture	Missing	Default `scope='team'` set in schema. No inference rule shipped.
Error and retry protocol	Missing	Sensing surfaces `perception_error` / `perception_write_error` in tool JSON. No exponential-backoff buffered retry yet.
Embedding model cold start	Partial	OSS embedding calls go to OpenAI/Voyage/Cohere — vendor warm by default. Cold-start mostly non-issue; explicit warm-up still missing.

PART 2 — Practical solution: build only what is missing

The revised design philosophy: tap into what already works, build only what is genuinely missing. Sensing and Control are the two components that do not exist anywhere today. Everything else — structural knowledge, memory storage, retrieval — is solved by existing tools. The developer experience is a two-MCP-server install (one in OSS — see Part 3).

Design philosophy

GitHub Copilot and Cursor already solve structural code knowledge well. Graphiti and Zep already solve temporal memory storage well. Mem0 already solves flat fact retrieval well. Building competing versions of these would be reinventing working wheels. The focus shifts entirely to what nobody has built: passive session observation and intelligent context injection.

What to build vs what to tap into

Concern	Solved by	Our role
Structural code knowledge (Localization)	Cursor index, GitHub Copilot Memory, LSP, Tree-sitter	Thin adapter layer — query whichever index the developer already has
Causal memory storage (Perception store)	pgvector for semantic search; temporal links via `decision_relations`.	Write the capture logic that decides what to store and how to structure it
Flat fact retrieval (Planning retrieval)	Mem0-style facts table + planning_blocks for inferred patterns.	Write the relevance scoring that ranks what to surface per task
Embeddings	`text-embedding-3-small` or equivalent. Already available, cheap, fast.	Call the API. Do not train.
Session observation (Sensing)	Nobody. This does not exist.	Build it. This is the primary genuine gap.
Context injection (Control)	Nobody. This does not exist.	Build it. This is the second genuine gap (cloud-only in v0.2.0).

Localization — resolved as adapter, not component

Localization is no longer a bespoke component in the build plan. It is an adapter interface with three intended backends. Sensing queries it; the developer never touches it directly.

Backend	When used	What Sensing gets	OSS status
Cursor local index	Developer is working in Cursor	File embeddings, symbol map, cross-file dependencies	Not built
GitHub Copilot Memory API	Developer uses Copilot in VS Code	Repository-level conventions and file structure	Not built

Backend	When used	What Sensing gets	OSS status
Files-touched-this-session map	Default — used by every editor including Claude Code	Set of file paths Claude has read/edited this session	**Shipped** — backs topic-shift Stage 2

Remaining design work (as of Part 2 cut)

- **Perception — causal capture logic.** Ship the four-step write pipeline; design contradiction handling.
- **Planning — relevance scoring.** Cloud-only; OSS approximates with recency + file-overlap.
- **Sharing — scope rules.** Schema column shipped; inference rules still TODO.

Perception component

Perception is where causal knowledge gets built. It receives decision signals from Sensing, decides what is worth writing, structures each entry as a causal graph node, and serves retrieval queries from the CLI (and, in cloud, Planning). The storage layer (Postgres + pgvector) already exists — what Perception builds is the logic that decides what to write and how to structure it.

Position in the system

Field	Value
Receives from	Sensing (decision signals + scores via `X-Project-Id`) · CLI (corrections via `POST /corrections`)
Sends to	CLI / Planning (retrieval results via `GET /decisions`); always-on summary via `GET /projects/:id/summary`
Stores in	Postgres (decisions, decision_relations) + pgvector embeddings
Key constraint	Must never block Sensing — all writes are async. Retrieval latency must be under 200 ms for CLI inject.

Causal graph entry — what gets written per decision

Every decision row has a clear shape. The `rejected[]` array is what distinguishes this from every other memory system — it stores what was considered and not chosen, with the reason each option was dropped.

Field	Type	What it captures
decision	TEXT	What was chosen, plain language
rationale	TEXT	Why — extracted from session turn, max 15 words
rejected	JSONB	Options considered but not chosen, each with reason
files_affected	TEXT[]	Files being discussed — from Sensing
scope	TEXT	user / local / team / global — defaults to 'team' in OSS
source	TEXT	`sensing | user_correction | init | claude_disagreement`
confidence	FLOAT	Classifier confidence 0–1 (defaults 0.8)
historical_relevance	FLOAT	Relevance score updated by Sensing's reply-scorer feedback
supersedes_id	TEXT	Links to a decision this one replaces
conflict_flag	BOOLEAN	Surfaces when this contradicts an active decision
needs_classification	BOOLEAN	True if captured from flush-on-close and pending reprocessing
invalidated_at	TIMESTAMPTZ	Set when no longer valid — never hard-deleted

Field	Type	What it captures
auto_resolved	BOOLEAN	True if conflict was auto-resolved (e.g. newer wins after 2 sessions)
reviewed_at	TIMESTAMPTZ	**New in OSS** — set when user explicitly approves in `robrain review`
embedding	vector(1536)	Semantic search vector; provider must match Sensing
timestamp / session_id / project_id	metadata	When written, which session, which project (always required)

Drift from original spec

Two fields shipped in OSS that the original spec did not list: `reviewed_at` (powers the new approval workflow in `robrain review`; see Part 3 visibility section) and `auto_resolved` (cloud-only logic, scaffolded in OSS schema). The spec's `scope` default of *'team'* ships, but no scope-inference rule runs at capture — every decision is `'team'` until a user edits it.

Write logic pipeline

- **Step 1 — confidence gate.** Signals below `DECISION_CONFIDENCE_MIN = 0.60` are discarded. Signals above `DECISION_CONFIDENCE_HIGH = 0.90` skip the contradiction check.
- **Step 2 — existing entry check.** Query Postgres + pgvector for related nodes on the same files and topic. Cosine similarity above 0.82 AND file overlap → contradiction path. Sim > 0.82 with no file overlap → new node + soft link. Sim < 0.82 → new node, no link.
- **Step 3 — field extraction (Haiku call).** Haiku returns structured JSON: `{decision, rationale, rejected: [{option, reason}], confidence}`. Runs async, ~300 ms.
- **Step 4 — write to Postgres.** Create the row with the embedding and any `supersedes_id` link. Schedule a debounced summary regeneration.

Reality in OSS (v0.2.0, May 2026)

Steps 1, 3, 4 ship in OSS. Step 2's contradiction-detection logic is cloud-only — the OSS schema has the columns (`conflict_flag`, `supersedes_id`, `decision_relations`) but Perception does not auto-populate them. Writes always succeed; conflicts only surface when the user edits in `robrain review`. Summary regeneration uses a 30 s trailing debounce (`REGENERATE_SUMMARY_DEBOUNCE_MS`) to coalesce `init-project`'s burst writes.

Contradiction handling (designed)

Type	Example	Resolution
Direct reversal	'We switched back to Redux'	Invalidate old edge. Write new node. Link new to old with `supersedes` edge. Full history intact.
Partial update	'Zustand but only in the cart module'	Narrow scope of old node. Write new node for the exception. Both nodes valid with different scopes.

Type	Example	Resolution
Ambiguous conflict	Classifier not sure if this reverses or extends the prior decision	Write new node with `conflict_flag = true`. Surface to user via Control at next task boundary. Auto-resolve after 2 sessions if no response — newer wins.

Planning component (cloud-only)

Planning is the decision-making brain between Perception's memory store and Control's injection mechanism. Its job: given what the developer is doing right now, what subset of stored memory is worth surfacing? It does not store anything. It does not inject anything. It ranks, filters, and hands a scored list to Control.

> **Drift from original spec**
>
> Planning is cloud-only in v0.2.0. The OSS substitute is `npx robrain inject`, which orders results by (a) semantic similarity from `context_system.search_decisions()` when `--query` is given, (b) file-overlap from `context_system.decisions_by_file_overlap()` when `--files` is given, or (c) recency otherwise. The four-signal weighted scorer (semantic 0.35 + file_overlap 0.30 + recency 0.20 + historical_relevance 0.15) is cloud-only. The `planning_blocks` and `mem0_facts` tables exist in the OSS schema as scaffolding but are not populated by any OSS code path.

Four internal layers (cloud — Mission → What → How → Method)

- **Mission** — persistent project goal, stored once, rarely changes.
- **What** — current task description from Sensing's task-boundary signal.
- **How** — explicit developer rules (Mem0). Forced-include / forced-exclude beats scoring.
- **Method** — inferred patterns (Letta-style memory blocks). Updated from feedback scores after each session.

Implementation notes — critical flags

> **Critical: EMBEDDING_PROVIDER must be identical in Sensing and Perception**
>
> Embedding models map text into a high-dimensional vector space. Different providers — and even different models from the same provider — produce vectors in completely different spaces. Cosine similarity across them is meaningless. Setting `EMBEDDING_PROVIDER=openai` in Perception while Sensing uses `voyage` will silently return wrong results. Same model string (`OPENAI_EMBEDDING_MODEL=text-embedding-3-small`) must match too — different dimensions cause pgvector to throw, which is easier to catch but still breaks the system.

In OSS the `.env` at the repo root is loaded by both `pnpm docker:up` and `robrain install --self-hosted`, so a single value drives both processes. The CLI loader follows shell-wins precedence: shell environment variables override the `.env` file. Document this expectation if you introduce additional env scopes.

> **Re-embedding on provider change**
>
> If decisions were already written to Postgres using one embedding provider and you switch providers, all existing embeddings are invalid. A one-time migration script must SELECT all rows in `context_system.decisions` with non-null `embedding`, re-embed each `decision` + `rationale` with the new provider, and UPDATE the column. Until done, similarity search returns wrong results for all historical decisions.

Phase 2 considerations — what the field has learned

- **Memory garbage collection & staleness detection** — knowledge bases go stale as the codebase evolves. A decision about `cart.ts` becomes stale when `cart.ts` is significantly rewritten — but nothing currently detects that. Phase-2 target: Localization adapter watches git diffs after each session.
- **Procedural memory** — the missing third type alongside episodic and semantic. 'When I say refactor, I mean extract functions.' OSS schema includes `fact_type='procedure'` in `mem0_facts` as a hook; no capture path yet.
- **Injection positioning within Claude's context** — 'lost in the middle.' Cloud Control will inject as a user-turn message rather than a tool result.
- **Second-pass reranking on Planning output** — Haiku reranker between Planning and Control. Cloud, post-instrumentation.
- **Multi-agent contradiction resolution** — extend the schema with an `agent_id` field; add an ownership map. Becomes relevant once parallel agents are common.
- **Within-session architectural drift** — Sensing checks new decisions against in-session decisions before routing. Same-session surfacing.

V1 amendment — veto preservation

This amendment was added after analysing Supermemory's production architecture. It addresses a structural flaw in the original v1 design that would have caused the system's most valuable data — rejected alternatives — to silently disappear through compression. The fix is in two files and is shipped in

OSS.

Before and after — always-on summary format

	Format	What survives compression
Before	Decision: Use Zustand for state management (cart re-render performance)	The choice survives. The vetoes are gone after one summary regeneration.
After (shipped)	Chose Zustand over Redux (re-renders) over MobX (team unfamiliar) because cart perf	Choice AND vetoes survive indefinitely. Same token count. Veto-resistant by format.

Reality in OSS (v0.2.0, May 2026)

Shipped. The OSS Perception summary prompt enforces 'Format: "Chose X over Y (reason)." Preserve rejected alternatives verbatim.' `max_tokens` raised from 120 to 150 to fit vetoes. Trigger: scheduled debounced after each `POST /signals`; also callable via `POST /projects/:id/regenerate-summary`.

Pre-launch testing — critical checks

Three failure modes that will quietly break the system before it gets a chance to prove its value. None are architectural — they don't require redesigning anything. But they are the things most likely to cause the system to fail in practice even if it works perfectly in theory.

Test 1 — MCP tool call reliability

The risk: Claude Code has its own reasoning about when to use tools and will skip `sensing_record_turn` calls when focused. A system that captures 70% of decisions is meaningfully worse than one that captures 95%. Pass threshold: ≥ 90% compliance on both `sensing_record_turn` and (cloud only) `control_inject_context`.

Test 2 — Haiku extraction false positive rate

The risk: a hallucinated decision stored at 0.65 confidence looks identical to a real one when injected six sessions later. Pass: ≤ 5% false positive rate. Add 3–5 few-shot negative examples if exceeded.

Test 3 — Cold start experience (sessions 1 and 2)

Developers judge new tools in the first two sessions. Solution shipped: `robrain init-project` reads CLAUDE.md, package.json, README, and recent `git log`; calls Haiku to infer 3–5 architectural decisions; writes them to Perception with `source='init'`; triggers a summary regeneration. Result: session 1 starts with a warm summary instead of a cold blank slate.

> **Drift from original spec**
>
> The original spec proposed a Control MCP tool `control_init_project`. In OSS this shipped instead as the `robrain init-project` CLI command (alias `robrain init`). The CLI form fits OSS better — there is no Control to host it — and the install command chains it by default (`--skip-init-project` to opt out).

Pre-launch checklist

#	Test	Status
1	MCP tool call compliance ≥ 90% across 10 sessions	OSS Sensing: instrumented via stderr logs. Compliance still depends on `CLAUDE.md` instructions; `init-project` writes them automatically.
2	Haiku false positive rate ≤ 5% across 50 turns	OSS uses basic prompt; calibrated prompt + few-shots is the cloud differentiator. Approval gate (`reviewed_at`) mitigates by keeping unapproved decisions out of the auto-export path.
3	Haiku false negative rate ≤ 15%	Implicit-decision proxies (imports, package-manager invocations) added to keyword scan to reduce miss rate.
4	Sessions 1–2 feel noticeably different from vanilla Claude Code	`init-project` shipped as default chain from `install`.
5	`control_init_project` built if test 4 fails	Replaced by `robrain init-project` CLI.

#	Test	Status
6	Few-shot negative examples added to prompt if test 2 fails	OSS basic prompt — pending. Cloud has the calibrated version.
7 (new)	`robrain review` after session 1 — decisions match what was discussed; vetoes preserved	Approve / Edit / Reject + `--approve-all` shipped. `--history` shows full lifecycle.

Autonomy stack inspiration — self-driving car lessons

The overall system design was inspired by the autonomy stack used in self-driving vehicles. Four structural lessons transfer directly. One is implemented in cloud v1. Three are design targets for Phase 2 and Phase 3.

AV layer	Component	Key lesson	Phase
Perception (cameras, lidar, radar)	Sensing + Localization	Sensor fusion — you currently have one sensor. Adapter adds a second.	Phase 2
World model (3D map)	Perception (causal graph)	World models need confidence decay. Recency does this for ranking, not for trust display.	Phase 2
Prediction	Missing entirely	AV systems predict before acting. Pre-load vetoes before Claude re-suggests rejected alternatives.	Phase 3
Planning	Planning API (cloud)	AV planning minimises a unified cost function. Define yours explicitly.	Phase 2
Control (steering, throttle, brake)	Control MCP (cloud)	Disengagement protocol — when to hand back to the human.	Cloud v1 ✓

V1 — disengagement protocol (cloud, shipped)

Parameter	Value	Meaning
DISENGAGEMENT_THRESHOLD	0.35 (default)	If Planning's top-ranked memory scores below this, Control stays silent rather than injecting.
DISENGAGEMENT_NOTIFY_AFTER	3 (default)	After 3 consecutive disengagements on the same project, Control surfaces a notification.
Disengagement log	stderr: '[Control] Disengaged'	Every disengagement logged with score, threshold, task description.
Reset on success	`disengagement_count` resets to 0	Notification only fires on sustained low confidence.

Developer review visibility — trust layer

A developer reviewing the system raised a critical concern: they want to see what is being stored before trusting it to inject into future sessions. This is correct instinct. The review layer is the trust layer — it is what separates a system developers adopt from one they disable after session 3.

Three-tier visibility design (with OSS status)

Tier	What it does	OSS status
Tier 1 — session-end summary	Plain-language summary returned by `sensing_end_session` / (cloud) `control_end_session`; surfaced directly by Claude.	OSS surfaces summary via the CLI flow; auto-summary in chat is cloud-only.
Tier 2 — `robrain review`	Per-decision Approve / Edit / Reject. Superseded decisions dimmed with lifecycle labels under `--history`.	**Shipped.** Plus `--approve-all`, `--session`, `--all`, `--limit`.
Tier 3 — web dashboard on roryplans.ai	Table view per project across sessions: edit/delete in-place, view vetoes, confidence, files, dates.	Phase 2, cloud-only.

Approval workflow — new in OSS

The OSS schema added a `reviewed_at` column on `decisions`. The default `robrain review` feed shows only *unreviewed* active decisions; once approved (or `--approve-all`'d), a decision falls out of the review queue but remains visible under `--history`. `robrain export-memory` only emits approved decisions by default — explicit `--include-unreviewed` to override. This makes the user the gate between 'captured' and 'surfaced to future sessions'.

> **Drift from original spec**
>
> The original three-tier visibility design framed review as a passive inspection surface. The shipped OSS design promotes review to an *approval workflow*: there is now a binary state (reviewed vs not) that downstream surfaces (the export-memory bridge especially) read. Architecturally, this means the spec's 'Planning reads `historical_relevance`' loop should also consider 'has this decision been approved?' as a first-class signal — not just confidence + recency. See follow-ups in Part 4.

PART 3 — Phase 0: OSS architecture (shipped state)

RoBrain ships as Apache 2.0 open source. The OSS strategy: open source the capture layer (Sensing, schema, CLI) and a basic Perception, protect the intelligence layer (calibrated extraction prompt, Planning scorer, cloud Control MCP with disengagement + pre-task warnings). The moat is intelligence, not access.

OSS vs cloud split — actual v0.2.0

Component	OSS (shipped)	Cloud only	Why split here
Sensing MCP	Full source	—	The passive capture differentiator. No value in protecting.
Perception API	Self-hosted (`packages/perception-self-hosted`) with basic Haiku extraction, embeddings, summary regen, /scores feedback	Calibrated extraction prompt, contradiction auto-detection, conflict_flag population, agent ownership map	Basic prompt ~80% accuracy. Calibrated ~95% — calibration is the moat within Perception.
DB schema	Full `packages/shared/schema.sql`	—	The `rejected[]` field is the structural differentiator. Open-sourcing it anchors the architecture publicly.
CLI	Full source — review, inject, explain, export-memory, projects, init-project, install, status, rule, logout	—	Trust layer must be open to be trusted.
Control MCP	**None.** Spec'd OSS Control MCP did not ship in v0.2.0.	Full Control MCP with auto-injection at task boundaries + disengagement + pre-task rejection warnings	OSS user paths to context flow through CLI inject / export-memory instead.
Planning API	—	Full 4-signal relevance scorer (semantic 0.35 + file_overlap 0.30 + recency 0.20 + historical_relevance 0.15) + Letta-block updates	OSS substitutes recency + file-overlap ordering inside CLI inject.
Synthesis	**Not yet shipped — README says 'Coming soon'**	Cloud roadmap	Three-pass scheduled job (cluster, contradiction scan, entity promotion).

Drift from original spec

Three deltas vs the original Phase 0 table: (1) the OSS Control MCP is not in v0.2.0 — it remains design intent only; OSS users use `robrain inject` + the new `robrain export-memory` bridge. (2) **Synthesis** is documented as 'Coming soon' in README — there is no `pnpm synthesis:run` command and no synthesis package; the three-pass design is preserved as roadmap. (3) Perception's veto-preserving summary regeneration **is** in OSS, with a 30 s trailing debounce that wasn't in the original spec.

Shipped surfaces

Sensing MCP tools (OSS, all four)

Tool	When called	Behavior
`sensing_start_session`	Session open	Returns `always_on_summary` fetched from Perception. Warm-starts session state.
`sensing_record_turn`	After every exchange	Returns immediately (buffer write synchronous). `topic_shift` is the only result computed inline. Sends `X-Project-Id` header on the async signal POST.
`sensing_end_session`	Session close	Triggers flush-on-close hook: 2 s grace window ships unclassified turns to Perception with `needs_classification` flag.
`sensing_get_status`	Debug / health	Returns buffer size, classifier queue, last signal timestamp.

Drift from original spec

All originally-spec'd `control_*` tools (`control_get_session_context`, `control_inject_context`, `control_get_context`, `control_record_correction`, `control_add_rule`, `control_end_session`) are **not in OSS**. Their OSS substitutes: `control_get_session_context` → Perception `GET /projects/:id/summary` (callable from CLI / cloud Control); `control_get_context` → `robrain inject --query / --files`; `control_record_correction` → Perception `POST /corrections`, exposed via `robrain review`; `control_add_rule` → `robrain rule --add`.

Perception HTTP endpoints (OSS)

Method + path	Purpose
`GET /health`	Liveness — returns `{status:'ok', db:'connected', mode:'oss-self-hosted'}`.
`POST /signals`	Ingest from Sensing. Confidence-gates, runs Haiku extraction, embeds, writes.
`GET /decisions`	Backs `robrain review`, `robrain inject`, `robrain explain`. Filters: `session_id, all, history, limit`.

Method + path	Purpose
`POST /scores`	Reply-scorer feedback from Sensing. Updates `historical_relevance`.
`POST /corrections`	User corrections from `robrain review`. Source field distinguishes `user_correction` vs `claude_disagreement`.
`POST /projects` · `GET /projects` · `POST /projects/merge`	Project lifecycle and recovery (backs `robrain projects list / merge`).
`GET /projects/:id/summary`	Always-on summary fetched by `sensing_start_session`.
`POST /projects/:id/regenerate-summary`	Trigger veto-preserving Haiku summary regeneration; debounced 30 s.

CLI commands (OSS, shipped in v0.2.0)

Command	What it does
`robrain install --self-hosted`	Wire Sensing MCP into editor; chains `init-project` by default (`--skip-init-project` opts out). Self-hosted mode points at `localhost:3001` and skips Rory Plans auth.
`robrain init-project` (alias `init`)	Reads CLAUDE.md, README, package.json, recent `git log`; calls Haiku to infer 3–5 decisions; warm-starts Perception with `source='init'`.
`robrain review [--all] [--history] [--approve-all] [--session id] [--limit n]`	Per-decision Approve / Edit / Reject. Default feed shows only unreviewed active decisions. `--history` shows full lifecycle including superseded.
`robrain inject [--query] [--files] [--all] [--copy] [--limit]`	Veto-preserving format ('Chose X over Y (reason)'). OSS path replaces automatic Control injection.
`robrain explain <file> [--why] [--copy]`	'Why does this code exist?' — semantic + file-overlap query. `--why` shows full rationale + rejected[].
`robrain export-memory [--dry-run] [--include-unreviewed] [--to dir]`	**New in v0.2.0** — projects approved decisions into Claude Code's auto-memory directory (`~/.claude/projects/<slug>/memory/`) so they load on session start with no paste. Bridges to Claude's native Auto-memory rather than fighting it.
`robrain projects list / merge <from> <to>`	Recover phantom project ids; merge sessions/decisions across project rows.
`robrain rule --add / --list / --remove`	Manage explicit Planning rules (`always_include | always_exclude | preference`). Writes to `mem0_facts`.
`robrain status`	Pings Perception health; counts sessions, decisions, rules; surfaces `perception_error` from Sensing logs.
`robrain logout`	Clear local credentials.

Docker — what runs in self-hosted

Service	Image / build	Port	Notes
postgres	`pgvector/pgvector:pg16`	5432 (configurable)	Volume `robrain_postgres_data`; `schema.sql` applied on first boot.
perception	Built from `docker/Dockerfile.perception`	3001 (configurable)	`OSS_MODE=true` hardcoded. Reads .env at repo root. Idempotent startup migrations for upgrades.

Decision lifecycle — invalidation, not deletion

Decisions are never hard-deleted. When a decision changes, the old row is marked `invalidated_at = now()` and linked to the new one via `supersedes_id`. The full chain of architectural evolution is always queryable.

The OSS `decision_relations` table currently supports four typed edges: `supersedes | extends | conflicts_with | related_to`. The originally-spec'd `depends_on` and `refines` edges (and `inferTypedLinks()` background pass that would populate them) are deferred to the cloud Synthesis layer.

PART 4 — OSS reality & architecture follow-ups

This section is new. It catalogues the deltas surfaced by shipping the OSS package and proposes architectural changes the original spec did not anticipate. Items are grouped by whether they are spec-doc corrections (things the doc said that aren't true) or architectural follow-ups (things the build surfaced that the spec should now reflect).

Spec-doc corrections (apply to the document itself)

#	Where	Original claim	OSS reality
1	Phase 0 OSS table — Control MCP	OSS Control MCP exists with 'recent decisions, file-overlap sorted.'	Not shipped. OSS users use `robrain inject` and `robrain export-memory` instead. Update Phase 0 to remove OSS Control row, add export-memory bridge.
2	Phase 0 OSS table — Synthesis	Synthesis ships in OSS via `pnpm synthesis:run`.	README says 'Coming soon'; no synthesis package; no script. Mark Synthesis section as roadmap-only in Phase 0.
3	MCP Tool Interface — Control tools	Six `control_*` tools listed as part of the shipped surface.	None of these MCP tools exist in OSS. Move table to a 'Cloud Control MCP — design intent' section.
4	Decision lifecycle — typed edges	Six edge types: supersedes | extends | conflicts_with | related_to | depends_on | refines.	OSS schema has four. `depends_on` + `refines` are Synthesis-deferred.
5	Bug 2 fix — server-side reply scoring	Reply scoring 'moved from Sensing to Perception's POST /scores'.	Term-match still computed in Sensing. Only the persistence + relevance-update moved to Perception. Update wording.
6	Decision schema fields	Six fields listed (decision, rationale, rejected, files_affected, scope, metadata).	OSS ships those plus `source`, `supersedes_id`, `conflict_flag`, `needs_classification`, `historical_relevance`, `auto_resolved`, `reviewed_at`. Update field table.
7	Cold start — `control_init_project`	New MCP tool in Control to be built if test 4 fails.	Shipped as `robrain init-project` CLI command, chained from `install`. Update Test 3 / pre-launch checklist row 5.
8	Localization adapter backends	Three backends: Cursor index, Copilot Memory API, Tree-sitter fallback.	Only the session-local files-touched map ships. Cursor/Copilot adapters are open follow-ups, not 'shipped backends.'

Architectural follow-ups surfaced by OSS work

> **F1 — Approval becomes a first-class signal**

`reviewed_at` shipped to power the OSS `robrain review` approval workflow, but the architecture treats it only as a UI filter. The cloud Planning scorer should add an *approval boost* alongside `historical_relevance`: an approved decision is qualitatively different evidence than an auto-extracted one. **Action:** add a fifth signal (`approval_state ∈ {unreviewed, approved, edited, rejected}`) to the Planning weight model. `export-memory` already filters on this; Control should too.

F2 — Bridge to Claude Auto Memory is a new injection path

`robrain export-memory` writes approved decisions into `~/.claude/projects/<slug>/memory/`. Architecturally, this is a third injection channel alongside (a) cloud Control auto-inject and (b) CLI manual paste. The system diagram should add it: 'Perception → file-system export → Claude native Auto-memory load.' Trade-off vs Control: zero latency, no MCP runtime, but stale until next export. **Action:** document the channel, decide whether to schedule periodic re-exports (cron / post-session hook).

F3 — Project ID stability

Project ID is currently derived from a hash of the working directory path. It does not survive `mv`, `cp -r`, or nested clones, and it fragments easily across machines. README already lists the fix as a follow-up: hash `git config --get remote.origin.url` (normalised), with a `.robrain/project.json` UUID fallback for non-git repos. **Action:** add a one-time migration in Perception that merges fragments by content fingerprint; teach `robrain projects merge` to suggest candidate merges.

F4 — OSS schema scaffolding for cloud-only features

`mem0_facts` and `planning_blocks` tables exist in the shared OSS schema but are never populated by an OSS code path. Same for `conflict_flag`, `auto_resolved`, `decision_relations`. Two architectural choices: (a) document them as 'cloud-feature scaffolding — present for forward-compat,' or (b) split `schema.sql` into `schema-core.sql` (OSS-used) and `schema-cloud.sql` (cloud-only). Recommendation: (a) — splitting the schema breaks `robrain rule --add` portability if a user later upgrades. Add a comment block in `schema.sql` calling out 'unused by OSS, populated by cloud Planning / Synthesis.'

F5 — Contradiction detection has schema but no logic in OSS

`conflict_flag`, `supersedes_id`, and `decision_relations` all ship in OSS, but no OSS Perception code populates them. The original spec presented contradiction handling as a Perception feature; in v0.2.0 it is implicitly cloud-only. **Action:** ship a basic OSS contradiction detector (cosine sim > 0.82 + file overlap → set `conflict_flag`; surface in `robrain review` as a 'review this' row). Cloud retains the calibrated reversal/extension/ambiguous Haiku classifier.

F6 — Synthesis is the load-bearing differentiator vs Auto Memory

Auto Memory captures and retrieves. README's positioning leans hard on Synthesis ('memory that compounds on its own') as the thing that separates RoBrain at the architecture level. Today Synthesis is 'Coming soon.' Architecturally this is the highest-leverage OSS gap — without Synthesis, RoBrain's claim is aspirational. **Action:** commit to Pass 1 (cluster + `planning_blocks` writeback) as the first OSS Synthesis ship, since it directly populates a table the schema already has. Defer Pass 2 (corpus contradiction scan) and Pass 3 (entity promotion) to follow.

F7 — Operational architecture: stale Docker image / migration drift

README documents that pulling new code without rebuilding the `perception` container leaves Postgres running an older binary, so idempotent startup migrations don't run. This is an architecture concern — the system depends on in-Perception migrations rather than versioned migration files. **Action:** introduce a `migrations/NNN_*.sql` directory, run on Perception boot in lexical order, with a `schema_migrations` tracking table. Closes the 'rebuild or break' footgun and lets future contributors ship migrations in PRs instead of conditional `ALTER`s in `index.ts`.

F8 — Cross-tool support is currently weaker than the README implies

README describes Cursor and Copilot setups; reality is Cursor needs manual `.cursorrules` wiring and there is no Copilot adapter. Architecturally the Localization 'adapter' is a single backend (session-local file map). **Action:** either soften the README claim ('best-effort for Cursor today; Copilot adapter on roadmap') or commit to the Cursor index reader as the first new Localization backend, queried with a 100 ms timeout fallback to the in-session map (the spec's defined fallback contract).

F9 — User-curated memory should be a first-class flow

The OSS path naturally separates auto-capture from user-approval. This is not just a UI choice — it changes the architectural shape: the *injection-eligible set* is now (decisions WHERE `invalidated_at IS NULL AND reviewed_at IS NOT NULL`) rather than all active decisions. **Action:** document this as the canonical injection-eligibility rule for both OSS export and cloud Control. Update the Trust Layer section and the Disengagement protocol (a *captured-but-unreviewed* decision should always disengage in the cloud Control path until reviewed).

Summary of changes vs the original PDF

This document keeps Parts 1 and 2 (inception + practical philosophy) essentially intact, with green callouts noting where OSS implementation matches and red callouts noting drift. Part 3 was rewritten to reflect what actually ships in v0.2.0 (Sensing MCP with four tools, Perception-self-hosted with nine HTTP endpoints, the four-edge `decision_relations` table, the ten OSS CLI commands including the new `export-memory`). Part 4 is new — eight spec-doc corrections and nine architectural follow-ups surfaced by the OSS build.

The most consequential follow-ups are **F1** (approval as a Planning signal), **F2** (export-memory as a third injection channel), and **F6** (Synthesis is the load-bearing differentiator and is not yet shipped). All three change the architecture diagram itself, not just wording.

Document status: Phase 1 (inception + practical) preserved · Phase 0 OSS architecture rewritten to reflect v0.2.0 · Architecture follow-ups added · Last updated: 2026-05-08 · Source: `github.com/adelinamart/robrain` @ `main`.